

RECONCILIATION SYSTEM — IMPLEMENTATION ROADMAP

For Programmer & Project Manager

TABLE OF CONTENTS

1. Project Overview
 2. Technology Stack Decision Matrix
 3. Phase 1: Foundation & Database (Weeks 1-2)
 4. Phase 2: Data Ingestion Layer (Weeks 3-4)
 5. Phase 3: Reconciliation Engine (Weeks 5-6)
 6. Phase 4: Cash Flow & Reporting (Weeks 7-8)
 7. Phase 5: API & Authentication (Weeks 9-10)
 8. Phase 6: Frontend Dashboard (Weeks 11-12)
 9. Phase 7: Bank Integrations (Weeks 13-15)
 10. Phase 8: Testing, QA & Security (Weeks 16-17)
 11. Phase 9: Deployment & DevOps (Weeks 18-19)
 12. Phase 10: Monitoring & Launch Prep (Weeks 20-21)
 13. Daily/Weekly Rituals
 14. Definition of Done per Phase
 15. Risk Register
-

1. PROJECT OVERVIEW

System Name (suggested)

ClearFlow or ReconPro or SettleTrack

What It Does

A financial reconciliation SaaS platform that:

- Accepts daily card collection data, bank statements, and TPV (POS) closing reports
- Automatically matches collections against bank clearings and TPV reports (3-way reconciliation)
- Calculates fees per institution and flags discrepancies
- Generates a daily morning report with uncleared items, actual balances, and projected cash flow
- Provides a cash flow forecast with settlement dates and fund availability

Target Users

- SMEs with multiple TPV terminals
 - Retail chains with multi-acquirer setups
 - Accounting firms managing client reconciliations
 - Franchises with centralized treasury
-

2. TECHNOLOGY STACK DECISION MATRIX

Layer	Option A (Recommended)	Option B (Alternative)	Decision Criteria
Backend	Python + FastAPI	Node.js + Express	FastAPI has native async, auto-generated OpenAPI docs, excellent for financial APIs
Database	PostgreSQL 15+	MongoDB	PostgreSQL for ACID compliance, complex queries, JSONB for flexible schemas
ORM	SQLAlchemy 2.0 + Alembic	Prisma / Drizzle	SQLAlchemy is Python-native, mature, supports migrations
Task Queue	Celery + Redis	RQ / Bull	Celery is battle-tested for scheduled reconciliation jobs
Scheduler	Celery Beat	APScheduler	Celery Beat integrates with the task queue
Frontend	React 18 + TypeScript + Tailwind	Vue 3 + Vuetify	React has larger ecosystem, better charting libraries
Charts	Recharts + D3.js	Chart.js	Recharts is React-native, highly customizable
Auth	Auth0 / Clerk	Firebase Auth / Custom JWT	Auth0 handles MFA, SSO, compliance out of the box
File Storage	AWS S3 / MinIO	Google Cloud Storage	S3 is the standard for statement file drops
Hosting	AWS (EC2/EKS/RDS)	Google Cloud / Azure	AWS has the most mature fintech infrastructure
Container	Docker + Docker Compose	Podman	Docker is the standard for dev/prod parity
CI/CD	GitHub Actions	GitLab CI	GitHub Actions has better marketplace for security scanning
Monitoring	Datadog / Sentry	New Relic / LogRocket	Datadog for infrastructure, Sentry for error tracking
Email/SMS	SendGrid + Twilio	AWS SES + SNS	SendGrid has better deliverability analytics

FINAL RECOMMENDED STACK

Backend:	Python 3.11+ FastAPI SQLAlchemy 2.0 Alembic Pydantic v2
Database:	PostgreSQL 15 Redis (cache + Celery broker)
Frontend:	React 18 TypeScript Tailwind CSS Recharts TanStack Query
DevOps:	Docker Docker Compose GitHub Actions AWS ECS Fargate
Monitoring:	Sentry (errors) Datadog (metrics) PgHero (DB)
Security:	Auth0 AWS KMS (encryption) Vault (secrets)

PHASE 1: FOUNDATION & DATABASE (Weeks 1-2)

Goal

Set up the project skeleton, database schema, and core models with proper migrations.

Week 1 Tasks

Day 1-2: Project Setup

- Initialize Git repository with branch protection rules (main, develop, feature/*)
- Set up Python virtual environment (Poetry recommended over pip)
- Create monorepo structure:

```
/clearflow
  /backend
    /app
      /models
      /providers
      /services
      /api
      /core
      /migrations
    /tests
  Dockerfile
  pyproject.toml
  /frontend
    /src
    /public
  Dockerfile
  package.json
  /infra
    docker-compose.yml
    /terraform (optional)
  README.md
  .github/workflows/ci.yml
```

- Set up pre-commit hooks: black, ruff, mypy, pytest
- Create docker-compose.yml with PostgreSQL, Redis, and backend services

Day 3-4: Database Schema Design

- Design ERD (Entity Relationship Diagram) for:
 - institutions table
 - card_collections table
 - bank_movements table
 - tpv_closing_reports table
 - fee_structures table
 - reconciliation_results table

- `cash_flow_entries` table
- `morning_reports` table
- `users` / `tenants` / `workspaces` tables (for multi-tenancy)
- Add indexes on:
 - `card_collections(collection_date, institution_id, status)`
 - `bank_movements(value_date, institution_id, reference)`
 - `bank_movements(matched_collection_id)`
 - `tpv_closing_reports(report_date, terminal_id, batch_number)`
 - `reconciliation_results(collection_id, status, collection_date)`
- Add partial indexes for performance: `WHERE status = 'PENDING'`

Day 5: SQLAlchemy Models & Migrations

- Convert the dataclass models from the reference code to SQLAlchemy declarative models
- Use `Mapped[]` types (SQLAlchemy 2.0 style)
- Set up Alembic for migrations
- Create initial migration script
- Write seed data script for institutions and default fees

Week 2 Tasks

Day 6-7: Core Infrastructure

- Implement configuration management (Pydantic Settings with `.env`)
- Set up logging (structlog for structured JSON logging)
- Implement database connection pooling (asyncpg for async PostgreSQL)
- Set up Redis connection for caching and Celery
- Implement base repository pattern (abstracts CRUD operations)

Day 8-9: Multi-Tenancy Foundation

- Decide on tenancy strategy: **Schema-per-tenant** vs **Row-level security**
 - **Recommendation:** Row-level security (RLS) with `tenant_id` column
 - Easier to scale globally, simpler backups
- Add `tenant_id` to all tables
- Implement middleware to inject `tenant_id` from JWT claims
- Create tenant onboarding flow (workspace creation)

Day 10: Testing Foundation

- Set up pytest with fixtures
- Create `conftest.py` with:
 - Database fixture (creates test DB, runs migrations, rolls back after test)
 - Redis fixture (flushes test DB)
 - Factory Boy factories for all models
- Write unit tests for models and repositories
- Achieve >80% coverage on models layer

Deliverables

- Running `docker-compose up` spins up full local stack

- `pytest` passes with >80% coverage
 - Alembic migrations run successfully
 - Seed data creates default institutions and fees
-

PHASE 2: DATA INGESTION LAYER (Weeks 3-4)

Goal

Build all upload endpoints and file parsers for collections, bank statements, and TPV reports.

Week 3 Tasks

Day 11-12: File Upload Infrastructure

- Implement S3/MinIO file storage service
- Create file validation (type, size, virus scanning with ClamAV)
- Implement chunked upload for large CSV/Excel files
- Create upload job tracking table (`file_uploads` with status: PENDING, PROCESSING, COMPLETED, FAILED)

Day 13-14: CSV/Excel Parsers

- Implement `CollectionParser`:
 - Supports configurable column mapping (user maps their CSV columns to our schema)
 - Handles multiple date formats
 - Validates amounts (no negatives for collections unless refunds)
 - Deduplicates by reference + date + institution
- Implement `BankStatementParser`:
 - Supports CSV, CAMT.053 XML, MT940 text formats
 - Auto-detects format from file extension/content
 - Maps bank-specific reference formats
- Implement `TPVReportParser`:
 - Supports JSON, XML, CSV from major TPV providers
 - Validates batch numbers and terminal IDs

Day 15: Ingestion API Endpoints

- `POST /api/v1/collections/upload` — CSV/Excel upload
- `POST /api/v1/collections/manual` — Single manual entry
- `POST /api/v1/bank-statements/upload` — Bank statement import
- `POST /api/v1/bank-statements/manual` — Single manual entry
- `POST /api/v1/tpv-reports/upload` — TPV report upload
- `POST /api/v1/tpv-reports/manual` — Single manual entry
- All endpoints return a `job_id` for async processing

Week 4 Tasks

Day 16-17: Async Processing with Celery

- Create Celery tasks:

- `process_collection_upload(job_id)`
- `process_bank_statement_upload(job_id)`
- `process_tpv_report_upload(job_id)`
- Implement row-level error handling (bad rows go to `upload_errors` table)
- Add progress tracking (X of Y rows processed)
- Implement retry logic with exponential backoff for transient failures

Day 18-19: Data Validation & Enrichment

- Implement reference normalization (strip spaces, uppercase, remove special chars)
- Auto-detect institution from file metadata or user selection
- Validate that uploaded collections don't exceed TPV report totals (early warning)
- Cross-validate bank movements against known institution formats
- Implement duplicate detection (same reference + amount + date within 24h)

Day 20: Ingestion Testing

- Unit tests for all parsers with sample files
- Integration tests for upload → processing → database flow
- Load test: upload 10,000 rows in under 30 seconds
- Test error handling: corrupt files, wrong formats, missing columns

Deliverables

- All upload endpoints working with async processing
- Sample files for testing included in repo (`/backend/tests/fixtures/`)
- Upload errors are logged and retrievable via API
- Parser supports at least 3 date formats and 2 number formats

PHASE 3: RECONCILIATION ENGINE (Weeks 5-6)

Goal

Build the core 3-way matching engine with fuzzy logic and discrepancy detection.

Week 5 Tasks

Day 21-22: Matching Engine Core

- Implement `ReconciliationEngine` class (port from reference code)
- Strategy 1: Exact reference match + amount + date proximity
- Strategy 2: Fuzzy match (amount + date + institution) with scoring algorithm
- Strategy 3: Batch number exact match (TPV Collection)
- Strategy 4: Terminal ID + date match
- Implement match scoring threshold (configurable per tenant)
- Add `match_confidence` field (HIGH, MEDIUM, LOW)

Day 23-24: Fee Calculation Engine

- Port fee calculation logic from reference code

- Support all fee types: PERCENTAGE, FLAT, MIXED, TIERED
- Implement fee history (track fee changes over time)
- Add fee discrepancy detection:
 - Calculate expected fee
 - Compare with actual bank deduction (if inferable)
 - Flag if difference > threshold (e.g., €0.10 or 1%)

Day 25: Status Determination Logic

- Implement state machine for reconciliation statuses:

```
PENDING → PARTIAL → MATCHED → CLEARED
PENDING → UNMATCHED
MATCHED → DISCREPANCY → CLEARED (after manual review)
```

- Auto-transition rules (e.g., after 7 days unmatched → alert)
- Manual override capability (with audit trail)

Week 6 Tasks

Day 26-27: Discrepancy Detection

- Amount discrepancy: collection gross vs bank credit vs TPV net
- Date discrepancy: expected settlement date vs actual value date
- Fee discrepancy: calculated vs actual deduction
- Missing TPV report detection
- Missing bank clearing detection (collections with no bank match after N days)
- Implement discrepancy severity levels: WARNING, CRITICAL, INFO

Day 28-29: Reconciliation API & Background Jobs

- POST /api/v1/reconciliation/run — Trigger reconciliation (with date range)
- GET /api/v1/reconciliation/results — Query results with filters
- POST /api/v1/reconciliation/{id}/resolve — Manual resolution endpoint
- Celery Beat schedule: run reconciliation every 6 hours + every morning at 6 AM
- Implement reconciliation job locking (prevent concurrent runs for same tenant/date)

Day 30: Engine Testing

- Unit tests for each matching strategy
- Integration test: full 3-way match scenario
- Edge cases:
 - Partial match (bank yes, TPV no)
 - Multiple bank movements for one collection (split clearings)
 - One bank movement covering multiple collections (batch clearing)
 - Refunds and chargebacks
- Performance test: reconcile 1,000 collections in <10 seconds

Deliverables

- Reconciliation engine runs automatically on schedule

- 3-way matching works with >95% accuracy on test data
 - Discrepancies are flagged with clear reasons
 - Manual resolution is audited (who, when, why)
-

PHASE 4: CASH FLOW & REPORTING (Weeks 7-8)

Goal

Build cash flow forecasting and the morning report generation system.

Week 7 Tasks

Day 31-32: Cash Flow Engine

- Implement `CashFlowEngine` (port from reference code)
- Calculate actual balances from bank movements
- Project settlements based on:
 - Institution settlement delay (D+1, D+2, D+7)
 - Fee deductions
 - Weekends/holidays adjustment (banks don't settle on weekends)
- Generate day-by-day cash flow timeline
- Implement "availability calendar" (when will pending funds be available)

Day 33-34: Morning Report Generator

- Implement `MorningReportGenerator` (port from reference code)
- Sections:
 1. Reconciliation summary (matched/partial/unmatched/discrepancy counts)
 2. Uncleared collections grouped by institution
 3. Cash position (actual balance + projected 7d + projected 30d)
 4. Fee summary + discrepancies
 5. Alerts (anomalies requiring attention)
 6. Detail tables (top 20 uncleared items, recent discrepancies)
- Support multiple output formats: JSON, HTML, plain text, PDF

Day 35: Report Scheduling & Delivery

- Celery Beat: generate report every day at 7:00 AM local time
- Per-tenant timezone support
- Delivery channels:
 - Email (HTML + PDF attachment)
 - In-app notification
 - Webhook (for integrations with Slack, Teams, ERP systems)
 - API endpoint to fetch latest report

Week 8 Tasks

Day 36-37: Dashboard Data APIs

- GET `/api/v1/dashboard/summary` — Today's numbers at a glance

- GET /api/v1/dashboard/cash-flow — Cash flow timeline data
- GET /api/v1/dashboard/uncleared — Paginated uncleared items
- GET /api/v1/dashboard/discrepancies — Active discrepancies
- GET /api/v1/dashboard/institution-breakdown — Per-institution metrics
- All endpoints support date range filtering and pagination

Day 38-39: Historical Reporting

- Monthly reconciliation report (PDF export)
- Institution performance report (clearing speed, fee accuracy)
- Audit trail report (all manual resolutions, fee changes)
- Custom report builder (user selects fields, date range, format)

Day 40: Reporting Testing

- Unit tests for cash flow calculations
- Integration tests for morning report generation
- Verify report accuracy against known test scenarios
- Test email delivery with Mailtrap/SendGrid sandbox

Deliverables

- Morning report generates automatically and delivers via email
- Cash flow forecast is accurate within 1 day for confirmed settlements
- Dashboard APIs serve data in <500ms
- Reports are exportable to PDF and Excel

PHASE 5: API & AUTHENTICATION (Weeks 9-10)

Goal

Build a secure, documented REST API with multi-tenant authentication.

Week 9 Tasks

Day 41-42: Authentication & Authorization

- Integrate Auth0 or Clerk for identity management
- Implement JWT validation middleware
- Role-based access control (RBAC):
 - OWNER — Full access, billing, user management
 - ADMIN — All data operations, can't delete workspace
 - ACCOUNTANT — View reports, upload files, can't change settings
 - VIEWER — Read-only access
- Implement API key authentication (for ERP integrations and automated uploads)
- Add MFA enforcement option for owners

Day 43-44: API Documentation & Versioning

- Auto-generate OpenAPI schema from FastAPI

- Publish Swagger UI at `/docs` and ReDoc at `/redoc`
- Implement API versioning (`/api/v1/`, `/api/v2/`)
- Write API usage guide for integrators
- Rate limiting: 100 requests/minute per API key, 1000 per authenticated user

Day 45: Webhooks

- Implement webhook system:
 - `reconciliation.completed` — Sent when reconciliation finishes
 - `discrepancy.detected` — Sent immediately when discrepancy found
 - `morning_report.ready` — Sent when report is generated
 - `bank_movement.matched` — Sent when collection clears
- Webhook signature verification (HMAC-SHA256)
- Webhook retry logic with exponential backoff
- Webhook delivery logs

Week 10 Tasks

Day 46-47: API Security Hardening

- Implement CORS policies (strict whitelist)
- Add request signing for sensitive endpoints
- Implement IP allowlisting for API keys
- Add SQL injection protection (SQLAlchemy parameterized queries — already done)
- Add XSS protection headers
- Implement audit logging for all data mutations
- GDPR compliance: data export (JSON dump of all tenant data) and deletion endpoints

Day 48-49: Performance Optimization

- Add Redis caching for dashboard summary endpoints (TTL: 5 minutes)
- Implement database query optimization (EXPLAIN ANALYZE on slow queries)
- Add connection pooling tuning
- Implement request pagination with cursor-based pagination for large datasets
- Add database read replicas for reporting queries (if needed)

Day 50: API Testing

- Contract testing with schemathesis (auto-generated from OpenAPI)
- Security testing with OWASP ZAP or Burp Suite
- Load testing with Locust (100 concurrent users)
- Integration tests for auth flows (login, token refresh, logout)

Deliverables

- Fully documented REST API with authentication
- Webhook system operational with signature verification
- API passes security audit (no critical vulnerabilities)
- Load test: 100 concurrent users, <2s response time

PHASE 6: FRONTEND DASHBOARD (Weeks 11-12)

Goal

Build a responsive web dashboard for uploading data, viewing reports, and managing settings.

Week 11 Tasks

Day 51-52: Frontend Setup

- Initialize React 18 + Vite project
- Set up TypeScript with strict mode
- Configure Tailwind CSS with custom theme (colors, typography)
- Set up React Router v6 with protected routes
- Configure TanStack Query (React Query) for server state management
- Set up Zustand or Redux Toolkit for client state
- Configure Axios with interceptors for JWT refresh

Day 53-54: Layout & Navigation

- Implement sidebar navigation:
 - Dashboard (home)
 - Collections (upload + list)
 - Bank Statements (upload + list)
 - TPV Reports (upload + list)
 - Reconciliation (results + discrepancies)
 - Cash Flow (timeline + forecast)
 - Morning Reports (history + subscribe)
 - Settings (institutions, fees, users, API keys)
- Implement top bar with tenant switcher, notifications bell, user menu
- Mobile-responsive layout (collapsible sidebar)

Day 55: Dashboard Home Page

- KPI cards: Total Collections, Cleared, Pending, Discrepancies
- Cash flow mini-chart (last 14 days)
- Uncleared items table (top 10, sortable)
- Recent alerts feed
- Quick upload buttons

Week 12 Tasks

Day 56-57: Data Management Pages

- Collections page: upload zone (drag-and-drop), table with filters, detail modal
- Bank Statements page: same pattern
- TPV Reports page: same pattern
- All tables: sorting, filtering, pagination, export to CSV
- Implement bulk actions (delete, reprocess)

Day 58-59: Reconciliation & Cash Flow Pages

- Reconciliation results page: filter by status, date, institution
- Detail view for each result: collection + bank + TPV side-by-side comparison
- Manual resolution modal (dropdown for status + notes field)
- Cash flow page: interactive timeline chart
 - Toggle between actual and projected
 - Hover for details (amount, source, expected date)
 - Institution filter
- Availability calendar view (heatmap by day)

Day 60: Settings & User Management

- Institution settings: add/edit institutions, configure settlement delays
- Fee configuration: add/edit fee structures, effective dates
- User management: invite users, assign roles, deactivate
- API keys: generate, revoke, view usage
- Notification preferences: email frequency, webhook URLs
- Workspace settings: name, timezone, currency, date format

Deliverables

- Dashboard is responsive (works on tablet and desktop)
- All CRUD operations work without page reload
- Upload progress is visible and cancellable
- Charts are interactive and exportable

PHASE 7: BANK INTEGRATIONS (Weeks 13-15)

Goal

Connect to real bank APIs for automatic statement imports.

Week 13 Tasks

Day 61-62: PSD2 / Open Banking Research

- Research PSD2 compliance requirements (EU)
- Register as AISP (Account Information Service Provider) or partner with one
 - **Recommended:** Use Plaid, Nordigen (GoCardless), or Tink instead of direct PSD2
 - These are “Banking-as-a-Service” providers that abstract PSD2
- Evaluate providers:
 - **Plaid:** Best for US + EU, expensive but reliable
 - **Nordigen (GoCardless):** Best for EU, free tier available, strong in Europe
 - **Tink:** Strong in Nordics, owned by Visa
 - **TrueLayer:** Good UK/EU coverage
 - **Salt Edge:** Global coverage, including Latin America and Asia
- **Decision:** Start with Nordigen for EU + Salt Edge for global expansion

Day 63-65: Nordigen Integration (EU Focus)

- Implement Nordigen API client:

- Create requisition (user consent)
- List institutions (banks)
- Build link for user to authenticate with their bank
- Fetch account metadata
- Fetch balances
- Fetch transactions (with date range)
- Map Nordigen transaction format to our **BankMovement** model
- Implement automatic daily sync (Celery task)
- Handle requisition expiration (90 days for PSD2)
- Store Nordigen credentials securely (AWS KMS)

Week 14 Tasks

Day 66-68: Salt Edge Integration (Global)

- Implement Salt Edge API client (similar pattern to Nordigen)
- Support for non-EU banks (US, UK, LATAM, Asia)
- Handle different transaction formats per country
- Implement fallback: if automatic sync fails, user can upload CSV

Day 69-70: TPV API Integrations

- Research TPV provider APIs:
 - RedSys (Spain): usually file-based (SFTP), some APIs
 - BBVA TPV: API available for merchants
 - Stripe: Excellent REST API
 - Adyen: Comprehensive API
 - SumUp: API for transactions
- Implement Stripe integration first (easiest, most documentation)
 - Fetch payouts (settlements)
 - Fetch balance transactions
 - Map to our models
- Implement generic webhook receiver for TPV providers

Week 15 Tasks

Day 71-73: File-Based Integrations (SFTP)

- Many banks and TPVs still use SFTP file drops
- Implement SFTP client service:
 - Connect to bank/TPV SFTP servers
 - Download new files (CAMT.053, MT940, CSV)
 - Parse and ingest automatically
 - Archive processed files
 - Handle encryption (PGP/GPG decryption)
- Implement secure credential storage (HashiCorp Vault or AWS Secrets Manager)

Day 74-75: Integration Testing

- Sandbox testing with Nordigen/Salt Edge sandbox banks
- Test Stripe test mode

- Test SFTP with local test server
- Test error handling: expired consent, wrong credentials, server down
- Test reconciliation accuracy with real bank data (anonymized)

Deliverables

- Users can connect their bank via Nordigen (EU)
 - Daily automatic bank sync is operational
 - Stripe TPV integration works end-to-end
 - SFTP file drop processing is automated
-

PHASE 8: TESTING, QA & SECURITY (Weeks 16-17)

Goal

Ensure the system is robust, secure, and ready for production.

Week 16 Tasks

Day 76-78: Comprehensive Testing

- Unit test coverage target: >85%
- Integration tests for all API endpoints
- End-to-end tests with Playwright:
 - User login → upload collection → run reconciliation → view report
 - Upload bank statement → verify matching
 - Configure fee → verify calculation
- Performance tests:
 - Upload 50,000 rows in <2 minutes
 - Reconcile 10,000 collections in <30 seconds
 - Dashboard loads in <1 second
- Chaos testing: kill database, kill Redis, verify graceful degradation

Day 79-80: Security Audit

- Run `bandit` (Python security linter)
- Run `safety` (dependency vulnerability scanner)
- Run OWASP ZAP automated scan on all API endpoints
- Penetration testing (hire external firm or use Bugcrowd)
- PCI-DSS self-assessment (if you store card data — you shouldn't)
 - **Critical:** You must NOT store full card numbers. Only last 4 digits.
- GDPR compliance audit:
 - Data minimization (only collect what's needed)
 - Right to erasure implemented
 - Data portability (export endpoint)
 - Consent management
 - DPA (Data Processing Agreement) with subprocessors

Week 17 Tasks

Day 81-82: Compliance & Legal

- Draft Terms of Service
- Draft Privacy Policy (GDPR compliant)
- Draft Data Processing Agreement (DPA)
- Register as data controller with local DPA (if in EU)
- Implement cookie consent banner (if using analytics)
- Set up GDPR data request workflow (export/delete requests)

Day 83-85: User Acceptance Testing (UAT)

- Recruit 3-5 beta users (SMEs with card collections)
- Provide onboarding guide
- Collect feedback via structured form:
 - Upload experience (1-10)
 - Report clarity (1-10)
 - Discrepancy detection accuracy (1-10)
 - Feature requests
- Fix critical bugs found during UAT
- Document known issues and workarounds

Deliverables

- Security audit report with no critical findings
 - GDPR compliance checklist completed
 - UAT feedback incorporated
 - Legal documents published
-

PHASE 9: DEPLOYMENT & DEVOPS (Weeks 18-19)

Goal

Deploy to production with CI/CD, monitoring, and disaster recovery.

Week 18 Tasks

Day 86-88: Infrastructure as Code

- Set up AWS account structure:
 - Production account
 - Staging account
 - Development account
- Write Terraform configurations:
 - VPC with public/private subnets
 - ECS Fargate clusters (backend + frontend)
 - RDS PostgreSQL (Multi-AZ for production)
 - ElastiCache Redis
 - S3 buckets (file storage + backups)

- Application Load Balancer
- Route 53 DNS
- CloudFront CDN (for frontend assets)
- Set up environment-specific configuration

Day 89-90: CI/CD Pipeline

- GitHub Actions workflow:
 - **On PR:** Run tests, lint, type check, security scan
 - **On merge to develop:** Deploy to staging
 - **On merge to main:** Deploy to production (with manual approval)
- Implement blue-green deployment (zero downtime)
- Database migration strategy:
 - Migrations run before app deployment
 - Backward-compatible migrations only
 - Rollback plan for each migration
- Set up automated database backups (daily snapshots + point-in-time recovery)

Week 19 Tasks

Day 91-93: Monitoring & Alerting

- Application monitoring:
 - Sentry for error tracking (alert on >10 errors/hour)
 - Datadog or CloudWatch for metrics (CPU, memory, DB connections)
 - PgHero for PostgreSQL performance
- Business monitoring:
 - Number of reconciliations per day
 - Average matching accuracy
 - Upload success rate
 - Report generation time
- Alerting rules:
 - Database CPU > 80% for 5 minutes
 - API error rate > 1% for 5 minutes
 - Celery queue depth > 1000 jobs
 - Disk space > 85%
 - SSL certificate expiry < 30 days

Day 94-95: Disaster Recovery

- Document RPO (Recovery Point Objective) and RTO (Recovery Time Objective)
 - **Target:** RPO < 1 hour, RTO < 4 hours
- Implement cross-region backup replication
- Test disaster recovery plan:
 - Simulate database failure → restore from backup
 - Simulate region failure → failover to secondary region
- Document incident response runbook

Deliverables

- Production environment live and accessible
 - CI/CD pipeline deploys to staging automatically
 - Monitoring dashboards operational
 - Disaster recovery tested and documented
-

PHASE 10: MONITORING & LAUNCH PREP (Weeks 20-21)

Goal

Prepare for public launch with onboarding, documentation, and support.

Week 20 Tasks

Day 96-97: Onboarding Flow

- Implement guided onboarding:
 1. Welcome screen
 2. Add your first institution
 3. Configure fee structures
 4. Upload first collection (with sample file)
 5. Upload first bank statement
 6. Run first reconciliation
 7. View first morning report
- Create interactive tooltips and help bubbles
- Add “Getting Started” video (2-3 minutes)

Day 98-99: Documentation

- User documentation (Help Center):
 - How to upload collections
 - How to connect your bank
 - How to read the morning report
 - How to resolve discrepancies
 - FAQ (20+ questions)
- API documentation (for developers):
 - Authentication guide
 - Endpoint reference with examples
 - Webhook integration guide
 - SDKs (Python, Node.js)
- Admin documentation:
 - Deployment guide
 - Configuration reference
 - Troubleshooting guide

Day 100: Launch Preparation

- Set up Stripe or Paddle for billing (if SaaS model)
- Configure pricing plans:

- **Free:** 1 institution, 100 transactions/month, email reports
- **Pro:** €49/month — 5 institutions, unlimited transactions, API access, webhooks
- **Enterprise:** €199/month — Unlimited, SLA, dedicated support, custom integrations
- Set up customer support:
 - Intercom or Crisp chat widget
 - Support email (support@yourdomain.com)
 - Knowledge base
- Prepare launch assets:
 - Landing page
 - Pricing page
 - Sign-up page
 - Demo video

Week 21 Tasks

Day 101-102: Soft Launch

- Launch to waitlist (if you have one) or beta users
- Monitor closely:
 - Error rates
 - User engagement (daily active users)
 - Support ticket volume
 - Conversion rate (free → paid)
- Daily standup to triage issues
- Fix critical bugs immediately
- Collect testimonials from early users

Day 103-105: Public Launch

- Announce on:
 - Product Hunt
 - LinkedIn
 - Relevant subreddits (r/accounting, r/smallbusiness)
 - Twitter/X
 - Industry forums
- Reach out to:
 - Accounting firms (partnership opportunities)
 - POS/TPV providers (integration partnerships)
 - Business consultants (referral program)
- Launch PR campaign (optional, if budget allows)

Deliverables

- Onboarding completion rate > 70%
- Documentation complete and published
- Billing system operational
- Public launch executed
- First 10 paying customers

DAILY/WEEKLY RITUALS

Daily Standup (15 minutes)

- What did you complete yesterday?
- What are you working on today?
- Any blockers?

Weekly Review (1 hour, every Friday)

- Review burndown chart vs. plan
- Demo completed features
- Discuss blockers and adjustments
- Update risk register

Bi-Weekly Architecture Review (1 hour)

- Review any significant technical decisions
- Ensure code quality standards are met
- Discuss performance or security concerns

Monthly Retrospective (2 hours)

- What went well?
- What could be improved?
- Action items for next month

DEFINITION OF DONE PER PHASE

Every task must meet these criteria before being considered “Done”:

1. **Code:** Written, reviewed, and merged to develop
2. **Tests:** Unit tests pass (>80% coverage), integration tests pass
3. **Documentation:** Code is documented, API changes are documented
4. **Security:** No new vulnerabilities introduced (scan passes)
5. **Performance:** Meets performance benchmarks for that layer
6. **Demo:** Feature is demoed to the team and approved

RISK REGISTER

Risk	Probability	Impact	Mitigation
Bank API changes break integration	Medium	High	Abstract bank integrations behind adapter pattern; monitor API changelogs
PSD2 consent expires frequently	High	Medium	Auto-renewal reminders; fallback to manual upload

Table 2 – continued

Risk	Probability	Impact	Mitigation
Reconciliation accuracy < 90%	Medium	High	Extensive testing with real data; manual override capability; ML-based matching v2
Data breach / security incident	Low	Critical	Encryption at rest and in transit; regular audits; incident response plan; cyber insurance
Performance degradation with scale	Medium	High	Load testing; horizontal scaling; database optimization; caching
Programmer leaves mid-project	Low	High	Documentation; code reviews; pair programming; no single point of failure
Compliance changes (GDPR, PCI)	Medium	Medium	Legal counsel; compliance monitoring; flexible architecture
Competitor launches first	Medium	Low	Focus on niche (multi-acquirer reconciliation); superior UX; local market expertise

ESTIMATED BUDGET (for reference)

Category	Monthly Cost (Launch)	Monthly Cost (Scale)
AWS Infrastructure	€200-500	€2,000-5,000
Auth0 (2,000 users)	€0-23	€100-500
SendGrid (10k emails)	€0	€20-100
Datadog/Sentry	€0-50	€200-500
Domain + SSL	€10	€10
Banking APIs (Nordigen/Salt Edge)	€0-100	€500-2,000
Total Infrastructure	€300-700	€3,000-8,000

KEY MILESTONES

Milestone	Target Date	Success Criteria
M1: Foundation Complete	Week 2	docker-compose up works; DB migrations run; tests pass
M2: Ingestion Live	Week 4	Can upload all 3 data types; async processing works

Table 4 – continued

Milestone	Target Date	Success Criteria
M3: Matching Engine Live	Week 6	3-way reconciliation works; discrepancies detected
M4: Reports Working	Week 8	Morning report generates automatically; cash flow forecast accurate
M5: API Secure	Week 10	Auth works; API documented; rate limiting active
M6: Dashboard Usable	Week 12	Beta users can complete tasks without help
M7: Bank Integrations	Week 15	At least 2 bank APIs connected; daily sync works
M8: Security Audit Pass	Week 17	No critical vulnerabilities; GDPR compliant
M9: Production Live	Week 19	Deployed to AWS; monitoring active; DR tested
M10: Public Launch	Week 21	First paying customers; support system operational

Document Version: 1.0 Last Updated: 2026-08-02 Next Review: Weekly